Software security for Scottish Glen

Software security suggestions and implementation

Samuel Rutherford

CMP417: Engineering Resilient Systems

BSc Ethical Hacking Year 4

2023/24

Note that Information contained in this document is for educational purposes.

Contents

1 CONTEXT

1.1 BACKGROUND

Protecting infrastructure within the energy sector is a critically important task that must be done for both hardware and software. ScottishGlen is a company within the energy sector which utilises many applications, software and systems to fulfil the demands that are required from the energy sector. Protecting these applications and ensuring they continue to run smoothly should be an utmost priority which is why preparing against potential cyberattacks should be evaluated (Barlow, 2023).

Employees within ScottishGlen have been receiving messages, threatening the company, from a hacktivist group. The nature of the attack is unknown however the CEO of ScottishGlen has requested that the security posture of the company is protected to make it more resilient to an upcoming attack while considering the technical and employee perspective.

One of the systems that Scottish Glen utilise which the hacktivists may choose to exploit is their Kerberos Authentication system. It is a system or router which creates a gateway for users and the internet. This is effective in preventing cybercriminals from accessing a private network, such as one found within a company (Fortinet, n.d.).

Kerberos is also extremely popular resulting in many well-known and documented vulnerabilities being identified by cybersecurity professionals. These vulnerabilities are documented within the Common Vulnerabilities and Exposures (CVE) database and is an incredibly effective way to identify and classify vulnerabilities while evaluating what causes them to occur within a system. This database will be referenced and used to identify numerous vulnerabilities for Kerberos surrounding the common vulnerability known as buffer overflow. Numerous suggestions about to how counteract this vulnerability and how it occurs shall be enumerated upon within this report.

1.2 VULNERABILITIES

Kerberos authentication systems can be vulnerable to many different common vulnerability classifications, these CVEs are tracked within databases and made viewable through certain websites such as CVE.org and mitre.cve. These entries include the date it was published and the CVSS (Common Vulnerability Scoring System) which is a decimal number between 1 and 10 indicating the severity of the vulnerability. Common Weakness Enumeration (CWE) is a classification method for grouping together vulnerabilities.

Upon researching and examining CVEs to do with Kerberos, the most common vulnerability apparent in the CVE utilised buffer overflow. Buffer overflow is a CWE which is widely known for its ability to allow hackers to gain unauthorised access to systems remotely. This is achieved due to coding errors found within the software. Buffer overflow operates by exploiting how a program temporarily holds data within its memory (specifically RAM) before it is moved to another data location, including between processes or between input and output devices (Wikipedia contributors, 2024). This method of holding

the data is known as a ‘buffer’ and is commonly used to improve performance, allowing computers to quickly access data.

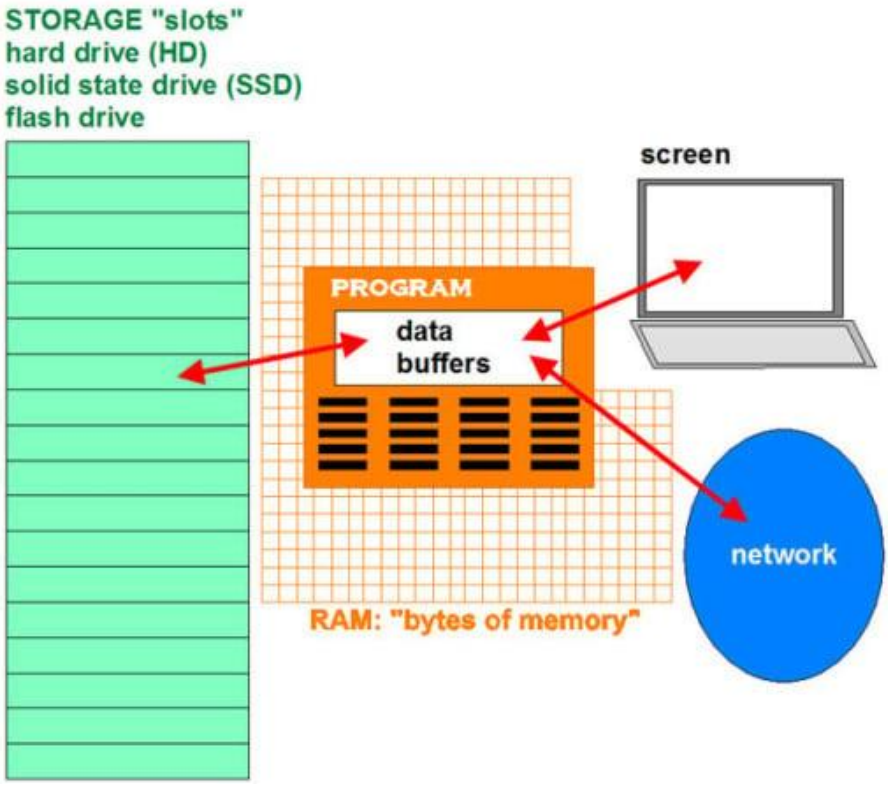


Figure 1–1: Diagram showing the interactions between data buffers and computer software and hardware (PCMag, n.d.).

As displayed within figure 1-1, the data from the program is held within the data buffers while it is being processed. In this example, the data could be instructions from the storage or network, which the program must follow. A critical issue with buffers is that they can be limited in size (with the exception for when they are used for video streaming services) resulting in data escaping the buffer and being released within the memory (Cloudflare, n.d.).

Buffer overflow occurs when the buffers capacity is overloaded from what is typically expected from the input vector of the program. Finding the limits of the buffer can be performed by fuzzing which involves sending large amounts of data to the input vector and observing the output.

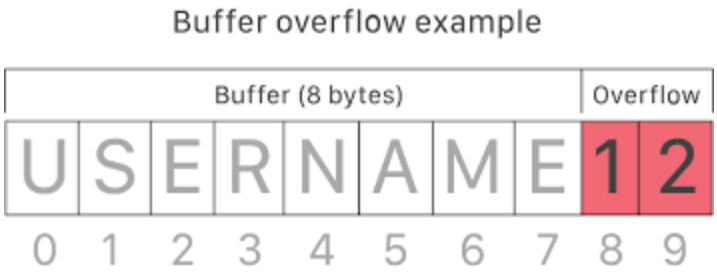


Figure 1-2: Example of an input value going beyond the buffer and causing overflow (figure from (Cloudflare, n.d.)*)*

When a cybercriminal performs a buffer overflow attack, they will try to go beyond the buffer and cause an error within the program handling the input vector for the buffer. Utilising the bytes outside the buffer within the input, the attacker would aim to make malicious changes to the programs execution paths or overwrite part of its memory. The result of this can lead to exposed files, system crashes and loss of control, which would be critically dangerous to an energy sector system. A generalised example of what could also be achieved within a buffer overflow attack could be replacing a segment in memory which contains a pointer with another pointer which points to a crafted payload, resulting in further exploitation of the application (Fortinet, n.d.).

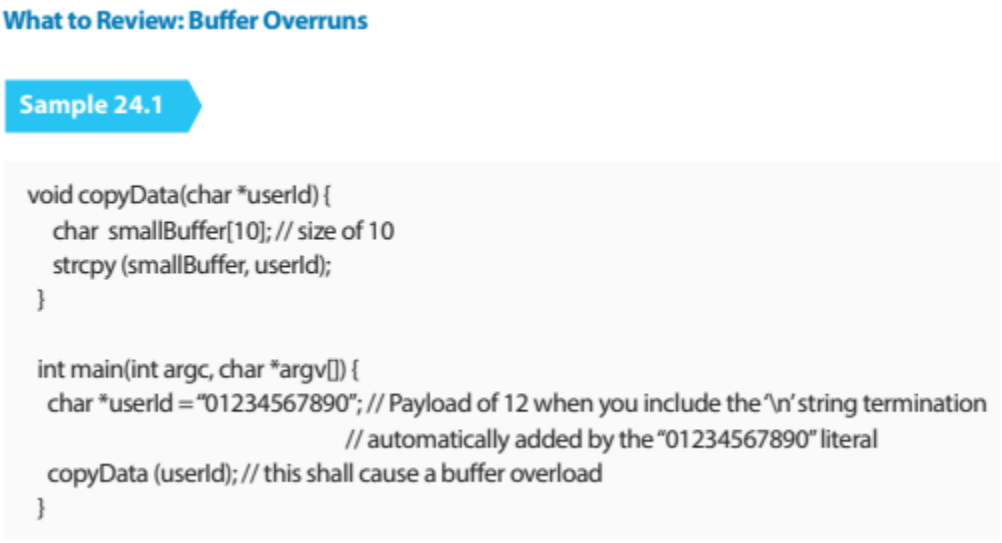
Heartbleed is a more specific example of buffer overflow being used to remove the privacy that the OpenSSL library provided. An attacker would have been able to send packets to the server which tricks the server into sending an excess amount of data based upon whatever the attacker claimed the buffer to be. However, OpenSSL never actually checked what the real size was, only using information from what the attacker had sent (and lied about) (Fruhlinger, 2022).

2 RECOMMENDATION

Buffer overflow is a very common vulnerability to be found within applications and systems however it can lead to extremely severe consequences for the company if the vulnerability is not patched, meaning that it should be within ScottishGlen's highest priority to identify these potential vulnerabilities found within their Kerberos authentication system and take preventative measures.

One of the issues with suggestion suitable preventative measures against the buffer overflow vulnerabilities present in Kerberos authentication is that ScottishGlen was not responsible for its development, being MIT (Massachusetts Institute for Technology), so active development recommendations would not be recommended such as secure coding practices. However, for preventing further vulnerabilities relating to buffer overflow with software, which is currently being developed, secure coding practices would be recommended, just not for this situation regarding Kerberos Authentication within ScottishGlen.

An effective recommendation to suggest which would help to mitigate the vulnerabilities would be code review for security. This is the practice which can utilise either manual or automatic processes to locate any already existing security vulnerabilities which arose during the development of the application or system (Synopsys, 2024). The code review should be performed by the team of developers or some of the technical staff, however hiring a third-party team to perform it is an option but more costly. Code review should follow guides or 'checklists' centred around security flaws found within the source code to cause vulnerabilities, such as buffer overflow, to arise. For example, the OWASP Code Review Guide v2 contains numerous coding errors which can easily show up during the development of the application, including ones for buffer overflow (however it is named "buffer overrun" within this guide). Within each section there are references of examples of code causing these vulnerabilities:



What to Review: Buffer Overruns

Sample 24.1

```
void copyData(char *userId) {
   char  smallBuffer[10]; // size of 10
   strcpy (smallBuffer, userId);
}

int main(int argc, char *argv[]) {
  char *userId = "01234567890"; // Payload of 12 when you include the '\n' string termination
                                // automatically added by the "01234567890" literal
  copyData (userId); // this shall cause a buffer overload
}
```

Figure 2–1: Examples of buffer overflow from OWASP Code Review Guide v2

Using examples such as the ones seen in figure 2-1, the team behind performing the code review should use this guide to form a checklist to look out for code which could cause the vulnerabilities. Often what is shown within the guide is an example of the vulnerability occurring, so rarely there will be an identical

match between code in the guide and the source code from the program being tested, meaning that the code reviewers must be knowledgeable in what is specifically causing the flaw.

Manual code review should be performed closely following the aforementioned guides and checklists. The technical team within ScottishGlen should aim to perform the manual code review on the Kerberos authentication application to identify where these coding flaws exist. An issue with manual code review is that it can be a draining process for the team and can be a slow process. OWASP estimates that a code reviewer should be able to review “100-200 lines of code per hour” assuming the reviewers has a sufficient understanding of the code and its design. One method for more active development code reviews would be Over-The-Should code reviews which involves another team member suggesting improvements which could be made to the code. This would be done while another team member is currently developing the code and aims to be a more informal approach as a downside of code review is that it can feel like audit and critique of the code writer’s performance.

Another method for performing secure code review is utilising tools to perform it automatically or greatly assist in collaboration between employees during development. A useful and free tool for automatic secure code review is Flawfinder which specifically analyses the source code for potential security flaws and outputs it in a readable format. Another automatic free tool would be American Fuzzy Lop (AFL) which operates differently to Flawfinder. While Flawfinder searches for source code errors causing vulnerabilities, AFL sends mass amounts of input data to the program checking if it causes errors with extremes or unexpected data, which could be effective with in identifying where the buffer overflow vulnerabilities are occurring. A paid alternative would be CodeScene which not only perform static code analysis but also behavioural analysis to identify social patterns which may cause continued risks within the code (Daityari, 2023).

3 IMPLEMENTATION

Using CVEdetail, there are 36 known vulnerabilities for Kerberos which have been documented, however there are 132 vulnerabilities recorded for Kerberos 5 and upon researching the specific vulnerabilities there appears to be overlap between the 2 software.

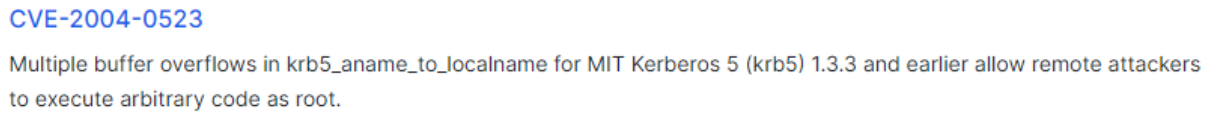


Figure 3–1: Image displaying an entry within CVEdetail which shows Kerberos 5, despite being filtered for just Kerberos.

For example, in figure 3-1, while specifically filtering for Kerberos vulnerabilities, there are mentions of the 5 version in the CVEs description. The CVE which will be CVE-2000-0392

3.1 CVE-2000-0392

CVE-2000-0392 is a buffer overflow vulnerability found within systems authenticated using Kerberos 4 or Kerberos 5 systems utilising certain utilities. It has been given a CVSS score of 7.2 and was published on 2000-05-16. This vulnerability can allow for local users to gain root privileges in the most severe scenarios. Within this CVE, there are 4 different buffer overflow vulnerabilities which all can lead to root privilege escalation.

Code responsible	Kerberos version
krb_rd_req()	5, 4
krb425_conv_principal()	5
krshd	5
ksu	5

Table 3–1: Table describing the functions or library responsible for the buffer overflow and version it is relevant to.

The krb_rd_req() function is used within nearly all Kerberos authentication services, however the actual buffer overflow occurs over 2 functions, firstly through krb_rd_req() and then that buffer is used to exploit krb425_conv_principal(). The exploit is able to achieve root administrator privileges due to certain daemons (such as krshd, rkinitd and kpopd) having setuid to root.

3.2 PATCHING AND MITIGATION

The number one priority for Scottish Glen should be to check the version of Kerberos that is running. The versions which found to be vulnerable to CVE-2000-0392 were:

- MIT Kerberos 5 releases krb5-1.0.x, krb5-1.1, krb5-1.1.1

- MIT Kerberos 4 patch 10, and likely earlier releases as well
- KerbNet (Cygnus implementation of Kerberos 5)
- Cygnus Network Security (CNS -- Cygnus implementation of Kerberos 4

The 2000 CERT Advisory suggests numerous solutions but primarily suggests downloading patches from either the vendor or MIT, however at the time the document was released, no vendor patches were available. Additionally, disabling version 4 authentication is suggested but priority should be patching.

Ideally, when the buffer overflow exploit within krb425_conv_principal() has been patched, that would result in krb_rd_req() also being fixed, however there is a chance that another function within krb_rd_req() could still cause another buffer overflow when used in conjunction. To mitigate this, secure code review is recommended before should be performed to evaluate any other functions which may lead to buffer overflow. The MIT Kerberos Team have a full advisory regarding the CVE which should be used in conjunction with the OWASP guidelines to create a checklist for the code reviewing team to follow (Software Engineering Institute, 2000).

To conclude, buffer overflow vulnerabilities should be patched immediately upon discovery but utilising efficient and secure code review and help to identify them in the first place. The hacktivists may aim to exploit a “zero-day” which refers to when a vulnerability does not have a patch but is known about (TrendMicro, n.d.). Secure code review could discover a potential zero-day within their code and assist MIT in patching to secure their own systems.

4 REFERENCES

Barlow, E. (2023, August). *A Surge of Cyber Security for the Energy Sector*. Retrieved February 23, 2024, from SecurityHQ: https://www.securityhq.com/blog/a-surge-of-cyber-security-for-the-energy-sector/

Cloudflare. (n.d.). *What is buffer overflow?* Retrieved February 22, 2024, from Cloudflare: https://www.cloudflare.com/learning/security/threats/buffer-overflow/

Daityari, S. (2023, September 26). *12 Best Code Review Tools for Developers*. Retrieved February 23, 2024, from Kinsta: https://kinsta.com/blog/code-review-tools/

Fortinet. (n.d.). *Buffer Overflow*. Retrieved February 22, 2024, from Fortinet: https://www.fortinet.com/uk/resources/cyberglossary/buffer-overflow

Fortinet. (n.d.). *Kerberos Authentication*. Retrieved February 22, 2024, from Fortinet: https://www.fortinet.com/uk/resources/cyberglossary/kerberos-authentication

Fruhlinger, J. (2022, September 6). *The Heartbleed bug: How a flaw in OpenSSL caused a security crisis*. Retrieved February 23, 2024, from CSO: https://www.csoonline.com/article/562859/the-heartbleed-bug-how-a-flaw-in-openssl-caused-a-security-crisis.html

PCMag. (n.d.). *buffer*. Retrieved February 22, 2024, from pcmag: https://www.pcmag.com/encyclopedia/term/buffer

Software Engineering Institute. (2000). *2000 CERT Advisories.* Software Engineering Institute. Retrieved from https://insights.sei.cmu.edu/library/2000-cert-advisories/

Synopsys. (2024, February 23). *Secure Code Review*. Retrieved from Synopsys: https://www.synopsys.com/glossary/what-is-code-review.html

TrendMicro. (n.d.). *Zero-Day Vulnerability*. Retrieved February 24, 2024, from TrendMicro: https://www.trendmicro.com/vinfo/gb/security/definition/zero-day-vulnerability

Wikipedia contributors. (2024, February 17). *Data buffer*. Retrieved Febuary 21, 2024, from Wikipedia: https://en.wikipedia.org/wiki/Data_buffer

5 BIBLIOGRAPHY

Barlow, E. (2023, August). *A Surge of Cyber Security for the Energy Sector*. Retrieved February 23, 2024, from SecurityHQ: https://www.securityhq.com/blog/a-surge-of-cyber-security-for-the-energy-sector/

Cloudflare. (n.d.). *What is buffer overflow?* Retrieved February 22, 2024, from Cloudflare: https://www.cloudflare.com/learning/security/threats/buffer-overflow/

CVE. (2024, February 22). *Index*. Retrieved from CVE: https://cve.mitre.org/index.html

CVEDetails. (2024, February 22). *Documentation*. Retrieved from CVEDetails: https://www.cvedetails.com/

Daityari, S. (2023, September 26). *12 Best Code Review Tools for Developers*. Retrieved February 23, 2024, from Kinsta: https://kinsta.com/blog/code-review-tools/

Fortinet. (n.d.). *Buffer Overflow*. Retrieved February 22, 2024, from Fortinet: https://www.fortinet.com/uk/resources/cyberglossary/buffer-overflow

Fortinet. (n.d.). *Kerberos Authentication*. Retrieved February 22, 2024, from Fortinet: https://www.fortinet.com/uk/resources/cyberglossary/kerberos-authentication

Fruhlinger, J. (2022, September 6). *The Heartbleed bug: How a flaw in OpenSSL caused a security crisis*. Retrieved February 23, 2024, from CSO: https://www.csoonline.com/article/562859/the-heartbleed-bug-how-a-flaw-in-openssl-caused-a-security-crisis.html

PCMag. (n.d.). *buffer*. Retrieved February 22, 2024, from pcmag: https://www.pcmag.com/encyclopedia/term/buffer

Software Engineering Institute. (2000). *2000 CERT Advisories.* Software Engineering Institute. Retrieved from https://insights.sei.cmu.edu/library/2000-cert-advisories/

Synopsys. (2024, February 23). *Secure Code Review*. Retrieved from Synopsys: https://www.synopsys.com/glossary/what-is-code-review.html

TrendMicro. (n.d.). *Zero-Day Vulnerability*. Retrieved February 24, 2024, from TrendMicro: https://www.trendmicro.com/vinfo/gb/security/definition/zero-day-vulnerability

Wikipedia contributors. (2024, February 17). *Data buffer*. Retrieved Febuary 21, 2024, from Wikipedia: https://en.wikipedia.org/wiki/Data_buffer